live-demo-script

ProPath — Live Demo Script

Speakers: Lovinson Dieujuste · Malcolm Richards
Driver (controls screen on every scene): Lovinson
Target runtime: ~7–8 minutes
Repo: github.com/Wisesofthemall/FocusFlow

How this script is structured

Each scene has **one designated speaker** who carries the whole scene as a continuous monologue. Lovinson is **always driving the screen** — every scene includes a parallel list of his screen actions that run underneath the speaker's narration.

Scene	Speaker	Screen driver
1 — Opening & team intro	**Lovinson**	Lovinson
2 — Auth guard	**Lovinson**	Lovinson
3 — Login	Malcolm	Lovinson
4 — Dashboard tour	**Lovinson**	Lovinson
5 — Applications list	Malcolm	Lovinson
6 — Create new application	Malcolm	Lovinson
7 — Persistence proof	**Lovinson**	Lovinson
8 — Tests & quality	**Lovinson**	Lovinson
9 — Close	Malcolm → Lovinson	Lovinson

Scene count: Lovinson 5, Malcolm 3, plus a split close (Malcolm recap → Lovinson handoff).

Pre-demo checklist (do this 5 minutes before going live)

Lovinson, before the audience joins:

#	Action	Why
1	Open **two terminals**, side by side, in `~/FocusFlow`. Label tab 1 `backend`, tab 2 `frontend`.	Avoids fumbling for the right tab on stage.
2	In `backend` tab, run `./mvnw spring-boot:run`. Wait for `Started ProPathApplication`.	Cold-start takes ~15s — never do this live.

#	Action	Why
3	In `frontend` tab, run `npm run dev`. Confirm `http://localhost:5173` is up.	Vite is fast, but doing it pre-show eliminates one risk.
4	Open Chrome with **three tabs** in this order: `localhost:5173`, `localhost:8080/h2-console`, the GitHub repo page.	The order matches the demo flow — no tab-hunting.
5	Sign **out** of any prior session so tab 1 lands on `/login`.	Scene 2 depends on the auth guard firing.
6	Zoom Chrome to **125%** (`Cmd +` + twice).	Code and form labels need to be readable on a projector.
7	Close Slack, email, notifications. Enable Do Not Disturb.	A toast popup mid-demo kills the vibe.
8	Have this script open on your **phone**, not the laptop.	Frees the screen for the demo itself.

SCENE 1 — Opening & team intro (~30s)

Speaker (full scene): Lovinson

> "Hey everyone, I'm Lovinson Dieujuste, and this is Malcolm Richards. We're team BlackCS, and we built **ProPath** — a career-prep app that helps students track job applications, surface relevant roles from a public jobs API, and stay organized through their career-prep journey. I'll be driving the screen for the rest of the demo, and Malcolm will jump in to walk through the architecture for the deeper technical scenes. What you're seeing on screen right now is our architecture diagram from the README on GitHub. Everything you're about to see is running entirely locally on this laptop — React frontend on port 5173, Spring Boot backend on 8080, talking to a file-based H2 database and the live RemoteOK jobs API. Let's get into it."

Lovinson's screen actions (in order):

1. Start on the **GitHub repo tab** (`Wisesofthemall/FocusFlow`).
2. Slowly scroll down to the README's architecture diagram. Hold it on screen for ~5 seconds while Malcolm talks.
3. As Malcolm wraps up, switch to the **localhost:5173 tab**. Audience now sees the login page sitting at `/login`.

SCENE 2 — Auth guard (~30s)

Speaker (full scene): Lovinson

> “Before we even log in, I want to point out one of the most important parts of the app — route protection. Every page in ProPath checks for a valid JWT before it will render. Watch what happens when I try to skip the login and go straight to the dashboard. I’m just typing the dashboard URL directly into the address bar — and the app immediately bounces me right back to the login page. On the frontend, that check lives in a `ProtectedRoute` component that reads from our `AuthContext`. But here’s the important part — even if a user faked that redirect by hacking the frontend code, the backend wouldn’t help them. Every `/api/**` route except `/api/auth/*` is gated by a Spring Security filter chain that validates the Bearer token on the server. So the auth check actually happens twice — once in the browser, once on the server. That’s defense in depth.”

Lovinson’s screen actions (in order):

1. Click the URL bar.
2. Type `localhost:5173/dashboard`.
3. Press **Enter**.
4. Audience sees the immediate redirect back to `/login`.
5. Pause 2–3 seconds so the redirect lands visibly.

SCENE 3 — Login with the seeded demo user (~40s)

Speaker (full scene): Malcolm

> “Now Lovinson is going to log in as our pre-seeded demo user. The backend seeds this account on first boot, so any grader who clones the repo can get straight in without having to register first. The email is `demo@propath.local` and the password is `Passw0rd!` — capital P, zero, exclamation point. When he hits **Sign in**, the frontend POSTs to `/api/auth/login`. The server pulls the user record, validates the bcrypt-hashed password, signs a JWT with our `JWT_SECRET`, and returns the token. Once that token comes back, the frontend stores it in `localStorage`, and an Axios interceptor attaches it as a `Bearer` header on every subsequent API request. That’s why every call from this point onward is automatically authenticated — Lovinson never has to think about the token, the interceptor does it for him. And we’re in. The dashboard you’re now looking at is what every authenticated user sees when they open the app.”

Lovinson’s screen actions (in order):

1. Click the **email field**. Type `demo@propath.local`.
2. Click the **password field**. Type `Passw0rd!`.
3. Click **Sign in**.
4. Land on the **Dashboard**.

> **If login fails:** the seed didn’t run. Stop the backend, `rm -rf backend/data`, restart `./mvnw spring-boot:run`, and try again. Have Malcolm fill the silence by walking through the architecture diagram on the GitHub tab.

SCENE 4 — Dashboard tour: three cards, three data sources (~60s)

Speaker (full scene): Lovinson

> "This is the dashboard you land on right after logging in. It pulls from three completely different places at once. The top card here — applications due this week — that's coming straight from our own database. The middle card is upcoming calendar events, also from our own database. And the bottom card is the interesting one — those are real, live job postings coming from the **RemoteOK** public API right now. The way we hit RemoteOK is worth explaining for a second. We don't call it from the browser. The frontend hits our own `/api/jobs/trending` endpoint, and Spring Boot uses a reactive `WebClient` to fetch from `remoteok.com`, then re-shapes the response before sending it back to us. That's deliberate — proxying through our own backend means we can cache responses, sanitize fields, and keep any future API key out of the browser entirely. It's the same pattern you'd use for any third-party integration in a real production app."

Lovinson's screen actions (in order):

1. Hover the **"Applications due this week"** card for ~2 seconds.
2. Hover the **"Upcoming events"** card for ~2 seconds.
3. Slowly scroll down to the **"Trending remote jobs"** card.
4. Let the audience read the card titles and roles for ~3 seconds.

SCENE 5 — Applications list (~40s)

Speaker (full scene): Malcolm

> "Lovinson is now clicking into the full Applications page. The three rows you see were seeded along with the demo user, so every grader who clones this repo sees the same starting state. Two things to point out about how this list works. First, it's sorted by urgency — soonest deadline first — so users always see what's most time-sensitive at the top of the page. Second, ownership is enforced server-side. If I logged in as a different user right now, this list would be completely empty. The JWT identifies the user, and our service layer filters every single query by that user ID — so users can never see each other's data, even if they hand-craft a malicious request to bypass the frontend. That's why the JWT in `localStorage` isn't just a convenience — it's the actual identity that the backend authorizes against on every request."

Lovinson's screen actions (in order):

1. Click **Applications** in the top nav.
2. Pause on the list view for ~3 seconds so the audience can read the rows.
3. Hover one row to highlight it. Don't click.

SCENE 6 — Create a new application (~60s)

Speaker (full scene): Malcolm

> "Lovinson is going to add a real application live so you can see the write path actually working. He's clicking **New application** now. When this form submits, it POSTs to `/api/applications` with a `CreateJobApplicationRequest` DTO. He's filling in the company — Pinterest — and the role, Software Engineer Intern. Status is **Applied**, deadline is sometime next month, and he's adding a quick note that this was submitted live during the demo. On the server side, when he hits **Submit**, the controller validates the DTO, the service stamps the current user from the JWT principal — that's how the system knows whose application this is — and JPA persists it to file-based H2. There's no mock data, no test fixtures involved here. The row that's about to show up on the next page is a real database write hitting actual disk. And there it is — back on the list, our Pinterest row just appeared in deadline order. That row exists in the database right now."

Lovinson's screen actions (in order):

1. Click + **New application**.
2. Click **Company** field, type `Pinterest`.
3. Click **Role** field, type `Software Engineer Intern`.
4. Select **Status: Applied**.
5. Pick any date next month for the **Deadline**.
6. Click **Notes**, type `Submitted live during demo`.
7. Click **Submit**.
8. Land back on the Applications list. The new Pinterest row should be visible.
9. Pause 3 seconds so the audience registers the new row.

SCENE 7 — Persistence proof: kill the backend, the data survives (~50s)

> **This is the scene that wins the grade.** It's the only way to prove H2 file-mode is doing what we say it is.

Speaker (full scene): Lovinson

> "Now the moment of truth. I'm going to kill the backend server completely, restart it from scratch, and then refresh the page. If we were using an in-memory database — which is the Spring Boot default and what most student projects use — that Pinterest row would disappear the second the server shuts down. Most projects accept that limitation. We deliberately chose not to. I'm switching to the backend terminal now, hitting Ctrl+C — the server is shutting down, you can see the shutdown logs scrolling by. Now I'm restarting it with `mvnw spring-boot:run`. While that boots back up, here's the architectural payoff: the data lives in a real file at `backend/data/propath.mv.db`. This same file-based H2 setup means we could swap to Postgres in production by changing two lines of `application.properties` — JPA gives us that database

portability for free. We're also not using Hibernate's `create-drop` mode, so the schema is preserved across restarts, not regenerated. That means a real production deployment with `update` mode or Flyway migrations is a small step away from where we are today. Server's back, I'm switching to the browser, refreshing the page — and the Pinterest row is still there. We just rebooted the entire backend, and the data we created twenty seconds ago survived. That's what real persistence looks like."

Lovinson's screen actions (in order):

1. Switch to the **backend terminal tab**.
2. Press **Ctrl+C**. Wait for shutdown logs to print.
3. Run `./mvnw spring-boot:run`.
4. Wait for `Started ProPathApplication` to print (~10 seconds).
5. Switch back to the **browser tab**.
6. Press **Cmd+R** (or F5) to refresh.
7. The Pinterest row is **still there**. Pause 3 full seconds.

   If the row disappeared: something is wrong with the data path. Don't panic — Lovinson should say "and that's the kind of regression our test suite is designed to catch," then jump to Scene 8.

SCENE 8 — Tests & quality (Milestone 4 evidence) (~50s)

Speaker (full scene): Lovinson

> "Last thing I want to show is quality. I'm opening a fresh terminal here and kicking off our backend test suite — that's `mvnw test`. While that runs, let me pull up the GitHub README to the Milestone 4 section so you can see our test summary. We have fifteen automated tests across both stacks — five backend tests and ten frontend tests — and they all pass. The backend tests are written in JUnit 5 with MockMvc, and the important thing is they exercise the **real** Spring Security filter chain. We deliberately did not mock the security layer out, because the whole point of testing auth is to make sure it actually works in production. So those tests prove that `/api/users/me` returns 200 with a valid JWT and 401 without one — same auth pipeline that runs in production is the one being tested. The frontend tests are in Vitest plus React Testing Library — they cover the auth guard, the login form including the failure path, and the status badge component for every possible application state. Our V8 statement coverage comes in at 62.92% across the files actually exercised by tests. And there's the green build right there — `Tests run: 5, Failures: 0, Errors: 0`. All passing."

Lovinson's screen actions (in order):

1. Switch to a free terminal tab and run `./mvnw test`.
2. While Maven boots (~5–10s), switch to the **GitHub README tab**.
3. Scroll to the **Milestone 4** section. Show the test summary table and the screenshots of green builds.
4. After ~20 seconds total, switch back to the terminal.

5. Audience sees `BUILD SUCCESS` and `Tests run: 5, Failures: 0, Errors: 0`.

   If you're tight on time: skip running the tests live. Just scroll to the README screenshots — they show the same green output.

SCENE 9 — Close (~20s)

Speaker: Malcolm (recap), then Lovinson (handoff to Q&A)

> **Malcolm:** "So to recap — React frontend, Spring Boot REST API with JWT-secured routes, persistent H2 database, a live external jobs API, fifteen passing tests, and a complete demo of the full stack working end to end."
>
> **Lovinson:** "All the code is on GitHub at `Wisesofthemall/FocusFlow`, and the full milestone documentation — the AI-usage log, the ERD, the endpoint table, the test evidence — is in the `docs/` folder of the repo. We're happy to take questions."

Lovinson's screen actions (in order):

1. Switch to the **GitHub repo tab**.
2. Scroll to the top so the team name and repo title are visible.
3. Hands off the keyboard. Take questions.

Q&A backup material (pull these up only if asked)

If asked about…	Show this
"How is the JWT validated?"	Open `backend/src/main/java/.../security/JwtAuthenticationFilter.java`
"What does the ERD look like?"	Open `docs/erd.md`
"Show me an example API request."	Open `docs/api-examples.md`
"What endpoints exist?"	Open `docs/endpoints.md`
"How did you use AI on this project?"	Open `docs/ProPath-Milestone-Document.md` §5.2

Failure recovery cheat sheet

Symptom	Fix
Login spins forever	Backend is down. Check the `backend` terminal tab. Restart with `./mvnw spring-boot:run`.

Symptom	Fix
Login returns 401 with the demo creds	Seed didn't run. Ctrl+C, `rm -rf backend/data`, restart backend.
Trending jobs card is empty	RemoteOK is rate-limiting. Malcolm should say "the public API occasionally rate-limits — the cache layer would handle this in production" and move on.
New application doesn't appear after submit	Hard refresh with `Cmd+Shift+R`. If still missing, click back into Applications from the nav to force a re-fetch.
Frontend shows a blank page	Vite HMR died. In the `frontend` tab: Ctrl+C, then `npm run dev`.
Tests fail live	Don't debug on stage. Malcolm should say "we'll re-run after the demo" and pivot to the README screenshots.